

Notebook : Translation to R – Pseudobulk Aggregation for Differential Expression

This notebook processes the preprocessed single-cell dataset (`adata_pp.h5ad`) from `single_cell_pipeline.ipynb`.

It performs pseudobulk aggregation (Macro-type × Donneur), applies statistical filters, and exports count matrices + metadata ready for differential expression analysis in R (limma/voom or DESeq2).

Pipeline Overview:

- Setup and Configuration
- Load Preprocessed Data and Validation
- Pseudobulk Aggregation (cell_type_annotation × donor_id)
- Statistical Filters (min cells per sample, min donors per cluster)
- Export for R Analysis
- Summary

Note: This pipeline does NOT perform differential expression. It prepares data for R-based analysis using established bulk RNA-seq methods (limma/voom, DESeq2).

Setup and Configuration

Environment Setup

Load required libraries and configure export directories for reproducible analysis.

In [1]:

```
import os
import numpy as np
import pandas as pd
import anndata as ad
import scanpy as sc
from pathlib import Path
from scipy.sparse import csr_matrix, issparse

import warnings
warnings.filterwarnings("ignore")

# Set random seed for reproducibility
np.random.seed(42)
```

```

# PROJECT_ROOT = "/Users/elodiehusson/Desktop/AD & PD" # Elodie
# PROJECT_ROOT = "C:/Users/yarad/Desktop/x/Masters/Master BMC -
Sorbonne/M2/Single Cell/Project/Coding Project" # Yara
PROJECT_ROOT = "C:/Z/AIDA_transcriptomics_project/transcriptomics-code" #
Laila

# Define directory structure
DIRS = {
    "DATA": os.path.join(os.path.dirname(PROJECT_ROOT), "data"),
    "EXPORTS": os.path.join(PROJECT_ROOT, "exports"),
    "TMP": os.path.join(PROJECT_ROOT, "tmp_cache")
}

# Create directories if they don't exist
for path in DIRS.values():
    os.makedirs(path, exist_ok=True)

os.chdir(PROJECT_ROOT)

# Define export directory as Path object
export_dir = Path(DIRS["EXPORTS"])
export_dir.mkdir(exist_ok=True)

# Define log function for pipeline progress
def log(message):
    """Display pipeline step messages"""
    print(f"📄 {message}")

log(f"Environment loaded. Working directory: {os.getcwd()}")
log(f"Exports will be saved to: {export_dir}")
📄 Environment loaded. Working directory:
C:\Z\AIDA_transcriptomics_project\transcriptomics-code
📄 Exports will be saved to:
C:\Z\AIDA_transcriptomics_project\transcriptomics-code\exports

```

Load Preprocessed Data and Validation

Load Preprocessed Dataset

Load the fully preprocessed AnnData object from `single_cell_pipeline.ipynb` > `add_cell_type_annotation.ipynb` and validate the presence of required metadata columns.

In [2]:

```

# Load the preprocessed dataset (output from single_cell_pipeline.ipynb)
dataset_path = os.path.join(DIRS["DATA"], "adata_annotated.h5ad")

```

```
adata = sc.read_h5ad(dataset_path)

log(f"Preprocessed dataset loaded: {dataset_path}")
log(f"Dataset dimensions: {adata.shape[0]:,} cells × {adata.shape[1]:,} genes")
```

 Preprocessed dataset loaded:
C:/Z/AIDA_transcriptomics_project\data\adata_annotated.h5ad

 Dataset dimensions: 62,800 cells × 2,000 genes

Validation and Integrity Checks

Verify the presence of required metadata columns and raw count data.

In [3]:

```
# Define required metadata columns
required_obs_columns = [
    'donor_id',
    'disease',
    'sex',
    'genetic_ancestry',
    'leiden',
    'cell_type_annotation' # macro-type validated
]

# Check presence of required columns
missing_columns = [col for col in required_obs_columns if col not in
adata.obs.columns]
if missing_columns:
    raise ValueError(f"❌ Missing required metadata columns:
{missing_columns}")

log(f"✅ All required metadata columns are present")

# Check presence of raw counts in layers
if 'counts' not in adata.layers:
    raise ValueError(f"❌ Raw counts not found in adata.layers['counts']")

log(f"✅ Raw counts found in adata.layers['counts']")

# Display dataset summary
print("\n" + "="*70)
print(f"

```

```
print(f"\nDonor distribution:")
print(adata.obs.groupby(['disease',
'donor_id']).size().unstack(fill_value=0))
print("=="*70 + "\n")
```

-   All required metadata columns are present
-   Raw counts found in adata.layers['counts']

DATASET SUMMARY

```
Number of cells: 62,800
Number of genes: 2,000
Number of Leiden clusters: 23
Number of macro-types (cell_type_annotation): 8
```

Donor distribution:

donor_id	Donor_31	Donor_228	Donor_333	Donor_545	\
disease					
dementia Alzheimer disease	2468	1436	4259	0	
dementia Parkinson disease	0	0	0	1896	
normal	0	0	0	0	

donor_id	Donor_609	Donor_614	Donor_634	Donor_638	\
disease					
dementia Alzheimer disease	0	2657	0	2385	
dementia Parkinson disease	0	0	0	0	
normal	6371	0	4031	0	

donor_id	Donor_721	Donor_735	Donor_865	Donor_941	\
disease					
dementia Alzheimer disease	0	0	5760	0	
dementia Parkinson disease	0	0	0	4532	
normal	7615	2710	0	0	

donor_id	Donor_1001	Donor_1007	Donor_1146	\
Donor_1278				
disease				
dementia Alzheimer disease	5760	576	0	
0				
dementia Parkinson disease	0	0	0	
0				
normal	0	0	6619	
3130				

donor_id	Donor_1467
disease	
dementia Alzheimer disease	0

```
dementia || Parkinson disease      595
normal                             0
```

Pseudobulk Aggregation (Macro-type × Donor)

Pseudobulk Construction

Aggregate raw counts by (cell_type_annotation, donor_id) to create pseudobulk samples.

Rationale: Cell-level differential expression violates statistical independence assumptions (pseudoreplication). Pseudobulk aggregation treats each donor as a biological replicate, enabling proper variance estimation in bulk RNA-seq methods (DESeq2, limma/voom).

In [4]:

```
# Create unique sample identifiers: cell_type_annotation__donor_id
adata.obs['sample_id'] = adata.obs['cell_type_annotation'].astype(str) +
"__" + adata.obs['donor_id'].astype(str)

log(f"Created sample_id column: {adata.obs['sample_id'].nunique()} unique
pseudobulk samples")

# Extract raw counts matrix (keep sparse format for memory efficiency)
counts_matrix = adata.layers['counts']

# Aggregate counts by sample_id using sparse matrix operations
sample_ids = adata.obs['sample_id'].values
unique_samples = np.unique(sample_ids)

# Create aggregation matrix (samples × cells)
from scipy.sparse import csr_matrix
sample_to_idx = {sample: idx for idx, sample in enumerate(unique_samples)}
row_indices = [sample_to_idx[sample] for sample in sample_ids]
col_indices = np.arange(len(sample_ids))
data = np.ones(len(sample_ids))

aggregation_matrix = csr_matrix(
    (data, (row_indices, col_indices)),
    shape=(len(unique_samples), len(sample_ids))
)

# Perform aggregation: (samples × cells) @ (cells × genes)
pseudobulk_counts_sparse = aggregation_matrix @ counts_matrix
```

```

# Convert to dense DataFrame (much smaller: 132 × 2000)
pseudobulk_counts = pd.DataFrame(
    pseudobulk_counts_sparse.toarray(),
    index=unique_samples,
    columns=adata.var_names
)

log(f"✅ Pseudobulk aggregation complete")
log(f"    Pseudobulk matrix dimensions: {pseudobulk_counts.shape[0]} samples
× {pseudobulk_counts.shape[1]} genes")

# Create metadata for pseudobulk samples
pb_metadata =
adata.obs.groupby('sample_id').first()[['cell_type_annotation', 'donor_id',
'disease']]

# Quality control: samples per cell type
print("\nSamples per cell type:")
samples_per_celltype =
pb_metadata.groupby('cell_type_annotation').size().sort_values(ascending=False)
for celltype, n_samples in samples_per_celltype.items():
    print(f"    {celltype}: {n_samples} samples")

# Quality control: samples per disease condition
print("\nSamples per disease condition:")
samples_per_disease =
pb_metadata.groupby('disease').size().sort_values(ascending=False)
for disease, n_samples in samples_per_disease.items():
    print(f"    {disease}: {n_samples} samples")

# Quality control: cross-tabulation cell type × disease
print("\nCross-tabulation (cell type × disease):")
crosstab = pd.crosstab(pb_metadata['cell_type_annotation'],
pb_metadata['disease'], margins=True)
print(crosstab)

# Quality control: donors per condition for each cell type
print("\nDonors per condition (minimum 3 recommended):")
for celltype in pb_metadata['cell_type_annotation'].unique():
    subset = pb_metadata[pb_metadata['cell_type_annotation'] == celltype]
    donors_per_condition = subset.groupby('disease')['donor_id'].nunique()
    print(f"\n{celltype}:")
    for condition, n_donors in donors_per_condition.items():
        status = "[OK]" if n_donors >= 3 else "[WARNING]"
        print(f"    {condition}: {n_donors} donors {status}")

```

📄 Created sample_id column: 132 unique pseudobulk samples

📄 ✅ Pseudobulk aggregation complete



Pseudobulk matrix dimensions: 132 samples × 2000 genes

Samples per cell type:

Inhibitory neuron: 17 samples
Excitatory neuron: 17 samples
Oligodendrocyte/OPC: 17 samples
Microglia: 17 samples
Support/Vascular/Immune: 17 samples
Astrocyte Reactive: 16 samples
Unknown/Low Quality: 16 samples
Astrocyte Homeostatic: 15 samples

Samples per disease condition:

dementia || Alzheimer disease: 61 samples
normal: 48 samples
dementia || Parkinson disease: 23 samples

Cross-tabulation (cell type × disease):

disease	dementia Alzheimer disease \
cell_type_annotation	
Astrocyte Homeostatic	6
Astrocyte Reactive	7
Excitatory neuron	8
Inhibitory neuron	8
Microglia	8
Oligodendrocyte/OPC	8
Support/Vascular/Immune	8
Unknown/Low Quality	8
All	61

disease	dementia Parkinson disease	normal	All
cell_type_annotation			
Astrocyte Homeostatic	3	6	15
Astrocyte Reactive	3	6	16
Excitatory neuron	3	6	17
Inhibitory neuron	3	6	17
Microglia	3	6	17
Oligodendrocyte/OPC	3	6	17
Support/Vascular/Immune	3	6	17
Unknown/Low Quality	2	6	16
All	23	48	132

Donors per condition (minimum 3 recommended):

Astrocyte Homeostatic:

dementia || Alzheimer disease: 6 donors [OK]
dementia || Parkinson disease: 3 donors [OK]
normal: 6 donors [OK]

Astrocyte Reactive:

```
dementia || Alzheimer disease: 7 donors [OK]
dementia || Parkinson disease: 3 donors [OK]
normal: 6 donors [OK]
```

Excitatory neuron:

```
dementia || Alzheimer disease: 8 donors [OK]
dementia || Parkinson disease: 3 donors [OK]
normal: 6 donors [OK]
```

Inhibitory neuron:

```
dementia || Alzheimer disease: 8 donors [OK]
dementia || Parkinson disease: 3 donors [OK]
normal: 6 donors [OK]
```

Microglia:

```
dementia || Alzheimer disease: 8 donors [OK]
dementia || Parkinson disease: 3 donors [OK]
normal: 6 donors [OK]
```

Oligodendrocyte/OPC:

```
dementia || Alzheimer disease: 8 donors [OK]
dementia || Parkinson disease: 3 donors [OK]
normal: 6 donors [OK]
```

Support/Vascular/Immune:

```
dementia || Alzheimer disease: 8 donors [OK]
dementia || Parkinson disease: 3 donors [OK]
normal: 6 donors [OK]
```

Unknown/Low Quality:

```
dementia || Alzheimer disease: 8 donors [OK]
dementia || Parkinson disease: 2 donors [WARNING]
normal: 6 donors [OK]
```

Build Pseudobulk Metadata

Construct metadata table with all required covariates for each pseudobulk sample.

In [5]:

```
# Build metadata DataFrame
metadata_columns = [
    'sample_id',
    'donor_id',
    'cell_type_annotation',
    'disease',
    'sex',
    'genetic_ancestry'
]
```

```

# Add cell_subtype if present, otherwise fill with NA
if 'cell_subtype' in adata.obs.columns:
    metadata_columns.insert(3, 'cell_subtype')

# Get unique metadata for each sample_id
pseudobulk_metadata =
adata.obs[metadata_columns].drop_duplicates(subset=['sample_id']).set_index
('sample_id')

# Add cell_subtype column if not present
if 'cell_subtype' not in pseudobulk_metadata.columns:
    pseudobulk_metadata['cell_subtype'] = 'NA'

# Calculate number of cells per sample
n_cells_per_sample = adata.obs.groupby('sample_id').size()
pseudobulk_metadata['n_cells'] = n_cells_per_sample

# Add sample_name column (identical to index)
pseudobulk_metadata.insert(0, 'sample_name', pseudobulk_metadata.index)

# Reorder columns to match specification
column_order = [
    'sample_name',
    'donor_id',
    'cell_type_annotation',
    'cell_subtype',
    'disease',
    'sex',
    'genetic_ancestry',
    'n_cells'
]
pseudobulk_metadata = pseudobulk_metadata[column_order]

log(f"✅ Pseudobulk metadata constructed")
log(f"    Total pseudobulk samples (before filtering):
{len(pseudobulk_metadata)}")

# Display sample distribution
print("\n📊 Pseudobulk sample distribution by cell type:")
print(pseudobulk_metadata.groupby('cell_type_annotation').size().sort_value
s(ascending=False))
print()
📋 ✅ Pseudobulk metadata constructed
📋    Total pseudobulk samples (before filtering): 132

📊 Pseudobulk sample distribution by cell type:
cell_type_annotation

```

Inhibitory neuron	17
Excitatory neuron	17
Oligodendrocyte/OPC	17
Microglia	17
Support/Vascular/Immune	17
Astrocyte Reactive	16
Unknown/Low Quality	16
Astrocyte Homeostatic	15

dtype: int64

Statistical Filters for Robustness

Apply Quality Filters

Apply two critical filters to ensure statistical robustness and synchronize count matrix:

1. **MIN_CELLS_PER_SAMPLE:** Remove samples with < 20 cells (insufficient for reliable aggregation)
2. **MIN_DONORS_PER_CLUSTER:** Remove cell types with < 5 unique donors (insufficient replication for DE)
3. **Count Matrix:** Synchronize counts matrix with filtered metadata

In [6]:

```
# FILTER 1: MIN_CELLS_PER_SAMPLE
MIN_CELLS_PER_SAMPLE = 20

# Store original metadata before filtering
pseudobulk_metadata_before_filter = pseudobulk_metadata.copy()

samples_before = len(pseudobulk_metadata)
pseudobulk_metadata = pseudobulk_metadata[pseudobulk_metadata['n_cells'] >=
MIN_CELLS_PER_SAMPLE]
samples_after = len(pseudobulk_metadata)

log(f"FILTER 1 (MIN_CELLS_PER_SAMPLE >= {MIN_CELLS_PER_SAMPLE}):")
log(f"    Samples removed: {samples_before - samples_after}")
log(f"    Samples retained: {samples_after}")

# Quality control: which cell types were affected by filter 1
if samples_before > samples_after:
    print("\nSamples removed per cell type:")
    removed_samples =
pseudobulk_metadata_before_filter[~pseudobulk_metadata_before_filter.index.
isin(pseudobulk_metadata.index)]
    removed_per_celltype =
removed_samples.groupby('cell_type_annotation').size().sort_values(ascendin
g=False)
```

```

        for celltype, n_removed in removed_per_celltype.items():
            print(f"    {celltype}: {n_removed} samples removed")
    else:
        print("\nNo samples removed by filter 1")

print("\nSamples retained per cell type:")
retained_per_celltype =
pseudobulk_metadata.groupby('cell_type_annotation').size().sort_values(ascending=False)
for celltype, n_retained in retained_per_celltype.items():
    print(f"    {celltype}: {n_retained} samples retained")

```

 FILTER 1 (MIN_CELLS_PER_SAMPLE >= 20):
 Samples removed: 31
 Samples retained: 101

Samples removed per cell type:

- Unknown/Low Quality: 11 samples removed
- Astrocyte Reactive: 9 samples removed
- Astrocyte Homeostatic: 6 samples removed
- Oligodendrocyte/OPC: 3 samples removed
- Microglia: 2 samples removed
- Excitatory neuron: 0 samples removed
- Inhibitory neuron: 0 samples removed
- Support/Vascular/Immune: 0 samples removed

Samples retained per cell type:

- Inhibitory neuron: 17 samples retained
- Excitatory neuron: 17 samples retained
- Support/Vascular/Immune: 17 samples retained
- Microglia: 15 samples retained
- Oligodendrocyte/OPC: 14 samples retained
- Astrocyte Homeostatic: 9 samples retained
- Astrocyte Reactive: 7 samples retained
- Unknown/Low Quality: 5 samples retained

In [7]:

```

# FILTER 2: MIN_DONORS_PER_CLUSTER
MIN_DONORS_PER_CLUSTER = 5

# Calculate number of unique donors per cell type
donors_per_celltype =
pseudobulk_metadata.groupby('cell_type_annotation')['donor_id'].nunique()

# Display donor count for all cell types (before exclusion)
print(f"\nDonors per cell type (threshold >= {MIN_DONORS_PER_CLUSTER}):")
for celltype, n_donors in
donors_per_celltype.sort_values(ascending=False).items():
    status = "[RETAIN]" if n_donors >= MIN_DONORS_PER_CLUSTER else
"[EXCLUDE]"
    print(f"    {celltype}: {n_donors} donors {status}")

```

```

# Identify cell types to exclude
celltypes_excluded = donors_per_celltype[donors_per_celltype <
MIN_DONORS_PER_CLUSTER].index.tolist()
celltypes_retained = donors_per_celltype[donors_per_celltype >=
MIN_DONORS_PER_CLUSTER].index.tolist()

# Filter metadata
samples_before = len(pseudobulk_metadata)
pseudobulk_metadata =
pseudobulk_metadata[pseudobulk_metadata['cell_type_annotation'].isin(cellty
pes_retained)]
samples_after = len(pseudobulk_metadata)

log(f"\nFILTER 2 (MIN_DONORS_PER_CLUSTER >= {MIN_DONORS_PER_CLUSTER}):")
log(f"    Cell types excluded: {len(celltypes_excluded)}")
if celltypes_excluded:
    log(f"    Excluded cell types: {celltypes_excluded}")
log(f"    Cell types retained: {len(celltypes_retained)}")
log(f"    Samples removed: {samples_before - samples_after}")
log(f"    Final samples retained: {samples_after}")

```

```

Donors per cell type (threshold >= 5):
    Inhibitory neuron: 17 donors [RETAIN]
    Excitatory neuron: 17 donors [RETAIN]
    Support/Vascular/Immune: 17 donors [RETAIN]
    Microglia: 15 donors [RETAIN]
    Oligodendrocyte/OPC: 14 donors [RETAIN]
    Astrocyte Homeostatic: 9 donors [RETAIN]
    Astrocyte Reactive: 7 donors [RETAIN]
    Unknown/Low Quality: 5 donors [RETAIN]

```



```
FILTER 2 (MIN_DONORS_PER_CLUSTER >= 5):
```



```
    Cell types excluded: 0
```



```
    Cell types retained: 8
```



```
    Samples removed: 0
```



```
    Final samples retained: 101
```

In [8]:

```

#
=====
==
# MANUAL EXCLUSION: LOW QUALITY CELLS
#
=====
==
# Rationale: Cells annotated as "Unknown/Low Quality" represent technical
# artifacts (doublets, apoptotic cells, low-quality captures) and must be
# excluded from differential expression analysis regardless of donor count.

```

```

#
# This exclusion was planned in add_cell_type_annotation.ipynb but not
# executed before saving adata_annotated.h5ad, resulting in these cells
# passing the automated MIN_DONORS filter (5 donors = threshold).
#
# Impact: Removes pseudobulk samples from "Unknown/Low Quality" category.
# Final analysis will proceed with 7 robust cell types.
#
=====
==

LOW_QUALITY_LABELS = ['Unknown/Low Quality', 'Unknown / Low Quality']

# Filter metadata to exclude low quality cells
samples_before_manual = len(pseudobulk_metadata)
pseudobulk_metadata = pseudobulk_metadata[
    ~pseudobulk_metadata['cell_type_annotation'].isin(LOW_QUALITY_LABELS)
]
samples_after_manual = len(pseudobulk_metadata)

# Remove unused categories from categorical column
pseudobulk_metadata['cell_type_annotation'] =
pseudobulk_metadata['cell_type_annotation'].cat.remove_unused_categories()

log(f"\nMANUAL EXCLUSION (Low Quality Cells):")
log(f"    Cell types excluded: {LOW_QUALITY_LABELS}")
log(f"    Samples removed: {samples_before_manual - samples_after_manual}")
log(f"    Samples retained: {samples_after_manual}")

# Display final cell type distribution (only retained cell types)
print("\nFinal cell type distribution:")
final_celltype_counts =
pseudobulk_metadata.groupby('cell_type_annotation').size().sort_values(ascending=False)
for celltype, n_samples in final_celltype_counts.items():
    print(f"    {celltype}: {n_samples} samples")

log(f"\n✅ Dataset ready for DGE analysis:")
log(f"    Total samples: {len(pseudobulk_metadata)}")
log(f"    Valid cell types:
{pseudobulk_metadata['cell_type_annotation'].unique()}")

```


MANUAL EXCLUSION (Low Quality Cells):


Cell types excluded: ['Unknown/Low Quality', 'Unknown / Low Quality']


Samples removed: 5


Samples retained: 96

Final cell type distribution:

```
Excitatory neuron: 17 samples
Support/Vascular/Immune: 17 samples
Inhibitory neuron: 17 samples
Microglia: 15 samples
Oligodendrocyte/OPC: 14 samples
Astrocyte Homeostatic: 9 samples
Astrocyte Reactive: 7 samples
```



✅ Dataset ready for DGE analysis:



Total samples: 96



Valid cell types: 7

In [9]:

```
# Synchronize counts matrix with filtered metadata
pseudobulk_counts = pseudobulk_counts.loc[pseudobulk_metadata.index]

log(f"\n✅ Filtering complete")
log(f"    Final pseudobulk matrix: {pseudobulk_counts.shape[0]} samples ×
{pseudobulk_counts.shape[1]} genes")

# Display final sample distribution
print("\n📊 Final pseudobulk sample distribution:")
print(pseudobulk_metadata.groupby('cell_type_annotation').size().sort_values(
ascending=False))
print()

📋
✅ Filtering complete
📋    Final pseudobulk matrix: 96 samples × 2000 genes

📊 Final pseudobulk sample distribution:
cell_type_annotation
Excitatory neuron          17
Support/Vascular/Immune    17
Inhibitory neuron          17
Microglia                  15
Oligodendrocyte/OPC        14
Astrocyte Homeostatic       9
Astrocyte Reactive         7
dtype: int64
```

Export for R Analysis

Export Files for Differential Expression in R

Generate four output files required for downstream analysis:

1. **pseudobulk_counts.csv**: Raw count matrix (genes × samples)
2. **pseudobulk_metadata.csv**: Sample metadata with covariates
3. **cluster_annotation_helper.csv**: Top marker genes per cluster
4. **adata_pseudobulk_info.h5ad**: Minimal AnnData object for reference

In [10]:

```
# Export 1: pseudobulk_counts.csv
# Format: genes (rows) × samples (columns)
counts_export = pseudobulk_counts.T # Transpose to genes × samples
counts_export_path = export_dir / "pseudobulk_counts.csv"
counts_export.to_csv(counts_export_path)

log(f"✅ Exported: {counts_export_path}")
log(f"   Dimensions: {counts_export.shape[0]} genes ×
{counts_export.shape[1]} samples")
📄 ✅ Exported: C:\Z\AIDA_transcriptomics_project\transcriptomics-
code\exports\pseudobulk_counts.csv
📄   Dimensions: 2000 genes × 96 samples
```

In [11]:

```
# Export 2: pseudobulk_metadata.csv
metadata_export_path = export_dir / "pseudobulk_metadata.csv"
pseudobulk_metadata.to_csv(metadata_export_path)

log(f"✅ Exported: {metadata_export_path}")
log(f"   Columns: {list(pseudobulk_metadata.columns)}")
📄 ✅ Exported: C:\Z\AIDA_transcriptomics_project\transcriptomics-
code\exports\pseudobulk_metadata.csv
📄   Columns: ['sample_name', 'donor_id', 'cell_type_annotation',
'cell_subtype', 'disease', 'sex', 'genetic_ancestry', 'n_cells']
```

In [12]:

```
# Export 3: cluster_annotation_helper.csv
# Extract top 10 marker genes per Leiden cluster from rank_genes_groups

if 'rank_genes_leiden' in adata.uns:
    # Extract marker gene results
    marker_results = []

    for cluster in adata.obs['leiden'].cat.categories:
        # Get top 10 genes for this cluster
        genes = adata.uns['rank_genes_leiden']['names'][cluster][:10]
        scores = adata.uns['rank_genes_leiden']['scores'][cluster][:10]

        for gene, score in zip(genes, scores):
            marker_results.append({
                'cluster': cluster,
                'gene': gene,
                'score': score
            })
```

```

marker_df = pd.DataFrame(marker_results)
marker_export_path = export_dir / "cluster_annotation_helper.csv"
marker_df.to_csv(marker_export_path, index=False)

log(f"✅ Exported: {marker_export_path}")
log(f"    Total markers: {len(marker_df)} (top 10 per cluster)")
else:
    log(f"⚠️ Warning: rank_genes_leiden not found in adata.uns")
    log(f"    Skipping cluster_annotation_helper.csv export")
📄 ✅ Exported: C:\Z\AIDA_transcriptomics_project\transcriptomics-
code\exports\cluster_annotation_helper.csv
📄    Total markers: 230 (top 10 per cluster)

```

In [13]:

```

# Export 4: adata_pseudobulk_info.h5ad
# Create minimal AnnData object with pseudobulk data

# Create new AnnData with pseudobulk counts
adata_pseudobulk = ad.AnnData(
    X=pseudobulk_counts.values,
    obs=pseudobulk_metadata,
    var=adata.var.copy()
)

# Clear unnecessary slots
adata_pseudobulk.obsm = {}
adata_pseudobulk.varm = {}
adata_pseudobulk.uns = {}
adata_pseudobulk.layers = {}

# Save with gzip compression
anndata_export_path = export_dir / "adata_pseudobulk_info.h5ad"
adata_pseudobulk.write_h5ad(anndata_export_path, compression='gzip')

log(f"✅ Exported: {anndata_export_path}")
log(f"    AnnData dimensions: {adata_pseudobulk.shape[0]} samples ×
{adata_pseudobulk.shape[1]} genes")
📄 ✅ Exported: C:\Z\AIDA_transcriptomics_project\transcriptomics-
code\exports\adata_pseudobulk_info.h5ad
📄    AnnData dimensions: 96 samples × 2000 genes

```

R Analysis Summary

In [14]:

```

# Generate final summary
print("\n" + "="*70)
print("----- SUMMARY -----")

```

```

print("="*70)
print(f"Pseudobulk samples generated: {len(pseudobulk_metadata)}")
print(f"Cell types retained (Macro):
{pseudobulk_metadata['cell_type_annotation'].nunique()}")
print(f"Clusters filtered/excluded: {len(celltypes_excluded)}")
if celltypes_excluded:
    print(f"    Excluded: {' ', ' '.join(celltypes_excluded)}")
print(f"\nExports written to: {export_dir}")
print("    └─ pseudobulk_counts.csv")
print("    └─ pseudobulk_metadata.csv")
print("    └─ cluster_annotation_helper.csv")
print("    └─ adata_pseudobulk_info.h5ad")
print("\n✅ PIPELINE COMPLETE – ready for R (limma/voom)")
print("="*70 + "\n")

```

```

=====
----- SUMMARY -----
=====

Pseudobulk samples generated: 96
Cell types retained (Macro): 7
Clusters filtered/excluded: 0

Exports written to: C:\Z\AIDA_transcriptomics_project\transcriptomics-
code\exports
    └─ pseudobulk_counts.csv
    └─ pseudobulk_metadata.csv
    └─ cluster_annotation_helper.csv
    └─ adata_pseudobulk_info.h5ad

```

```

✅ PIPELINE COMPLETE – ready for R (limma/voom)
=====

```